1. Semantic Rules Implemented

Using the (1–15) itemized list from the assignment specification, the following checklist identifies the implementation status of each semantic check.

1.1. Symbol Table Creation Phase

#	Rule	Implementation Details
1	Global scope table	SymbolTableBuilder.visit() creates SymbolTable("global", null) when it encounters the Prog AST node.
2	Class entries with local tables	processClassDecl() creates a class entry in the global table linked to a new class SymbolTable for each ClassDecl node.
3	Variable entries (data members, locals)	Data members added in processClassDecl() as "data" entries. Local variables added in processLocals() and processMain() as "local" entries.
4	Function entries with local tables	processFuncDef() creates function entries in the global table (free functions) or links them to class table declarations (member functions), each with a linked SymbolTable.
5	Duplicate detection and shadowing	Errors 8.1–8.4 (multiply declared) and warnings 8.5–8.6 (shadowing) are checked during table construction.
6	Declaration/definition consistency	checkMemberFuncConsistency() detects 6.2 (declared but not defined). processFuncDef() detects 6.1 (defined but not declared).
7	Symbol table output to file	SemanticDriver writes nested bordered tables to .outsymboltables using recursive printTable() at three indentation levels.
8	Multiply declared identifiers	8.1 (class), 8.2 (free function), 8.3 (data member), 8.4 (variable), 8.5 (shadowed inherited), 8.6 (local shadows class member).
9	Function overloading	9.1 (overloaded free function), 9.2 (overloaded member function), 9.3 (overridden inherited member function).

1.2. Semantic Checking Phase

#	Rule	Implementation Details
10	Type checking (expressions, assignment, return)	inferType() recursively infers expression types bottom-up. inferBinOp() checks 10.1. visitStatement() checks 10.2 (assignment) and 10.3 (return).
11	Undeclared identifier usage	11.1 (local variable), 11.2 (member variable via dot), 11.3 (member function via dot), 11.4 (free function), 11.5 (undeclared class type).
12	Function call parameters	checkCallParams() verifies parameter count (12.1) and types (12.2). Best-matching overload selected by parameter count.
13	Array dimension checking	inferDataMember() checks dimension count (13.1) and integer indices (13.2). checkCallParams() checks array parameter dimensions (13.3).
14	Circular class dependencies	checkCircularDeps() builds an inheritance graph and uses DFS with a recursion stack for cycle detection (14.1).
15	Dot operator validation	inferDot() verifies left operand is a class type (15.1) and resolves right operand in the class's symbol table.

2. Design

2.1. Overall Design

The semantic analyzer is implemented as a self-contained module in `src/SemanticAnalyzer/` containing five classes. It follows the Visitor design pattern as presented in the course lectures (Prof. Paquet, Week 7), where the AST from Assignment 3 is traversed twice by two independent concrete visitors.

Pipeline:

Source (.src) → Lexer → Parser → AST → `SymbolTableBuilder` (Pass 1) → `SemanticChecker` (Pass 2) → Output Files

2.2. Component Roles

Component	File	Role
`SymbolTable`	`SymbolTable.java`	Symbol table data structure. Each instance represents one scope (global, class, or function). Contains entries, a parent link for scope chain resolution, and inheritedTables for class inheritance. Lookups are case-insensitive.
`SymbolTableEntry`	`SymbolTableEntry.java`	One row in a symbol table. Fields: kind (class/function/data/param/local/inherit), name, type (normalized string or function signature), visibility, line number, and link (pointer to sub-table for class/function entries).
`SymbolTableBuilder`	`SymbolTableBuilder.java`	Pass 1 concrete visitor. Traverses the AST to build the complete symbol table hierarchy. Creates tables for classes, functions, and main. Detects declaration-level errors (6.1, 6.2, 8.1–8.6, 9.1–9.3, 14.1). Stores AST-to-table mappings in funcTableMap.
`SemanticChecker`	`SemanticChecker.java`	Pass 2 concrete visitor. Uses the completed symbol tables from Pass 1. Traverses function bodies with recursive inferType() for bottom-up type inference. Checks type errors (10.x), undeclared identifiers (11.x), function calls (12.x), arrays (13.x), and dot operator (15.1).
`SemanticDriver`	`SemanticDriver.java`	Driver/client. Runs ASTDriver for A1–A3 outputs, then runs Pass 1 and Pass 2 sequentially. Collects and sorts all errors by line number. Writes .outsymboltables and

		.outsemanticerrors files. Contains the symbol table printer.

2.3. Symbol Table Structure

The symbol tables form a nested hierarchy with three levels, connected by parent links (scope chain) and entry links (sub-tables):

- Global table → contains class entries (linked to class tables) and function entries (linked to function tables)
- Class tables → contain inherit entries, data entries, and function entries (linked to function tables when definitions are found)
- Function tables → contain param entries and local entries
- Scope chain: each table has a parent pointer. A member function's parent is its class table; a class table's parent is the global table. `lookupVar()` walks up this chain automatically.

2.4. Phase 1: Symbol Table Construction (SymbolTableBuilder)

Purpose: Build the complete symbol table hierarchy from the AST and detect all declaration-level semantic errors. This phase must complete before Phase 2 because type checking requires knowing what identifiers exist and their types.

Mapping of semantic actions to AST nodes:

AST Node	Method	Effect
`Prog`	`visit()`	Creates global SymbolTable. Processes ClassList, FuncDefList, ProgramBlock in order. Runs post-traversal cross-reference checks.
`ClassDecl`	`processClassDecl()`	Extracts class name from Id child. Checks duplicate class (8.1). Creates class entry in global table with linked class SymbolTable. Processes InherList (adds inherit entries, links parent tables). Processes MemberList (adds data and function entries).
`MemberDecl/VarDecl`	`processClassDecl()`	Extracts visibility, variable name, type, dimensions. Checks duplicate data member (8.3). Creates "data" entry in class table.
`MemberDecl/FuncDecl`	`processClassDecl()`	Extracts function name, builds signature string. Checks overloaded member function (9.2). Creates "function" entry in class table (link=null until definition found).

`FuncDef (free)`	`processFuncDef()`	Detected by Epsilon scope specifier. Checks duplicate (8.2) or overload (9.1). Creates function entry in global table with linked SymbolTable. Processes params and locals.
`FuncDef (member)`	`processFuncDef()`	Detected by Id scope specifier (class name). Finds matching unlinked declaration in class table. Links it to new function SymbolTable. Reports 6.1 if no match. Processes params and locals.
`FParam`	`processParams()`	Creates "param" entry in function table for each parameter.
`VarDecl (in FuncBody)`	`processLocals()`	Creates "local" entry. Checks duplicate (8.4). Checks class member shadowing (8.6) for member functions.
`ProgramBlock`	`processMain()`	Creates "main" function entry. Processes VarDecl children as locals. Checks duplicate (8.4) and undeclared class type (11.5).
`(post-traversal)`	`checkMemberFuncConsistency()`	Scans all class function entries. Reports 6.2 if link is null (declared but never defined).
`(post-traversal)`	`checkInheritance()`	Compares class members against inherited members. Reports 8.5 (shadowed data) and 9.3 (overridden function).
`(post-traversal)`	`checkCircularDeps()`	Builds inheritance graph. DFS cycle detection reports 14.1.

2.5. Phase 2: Semantic Checking (SemanticChecker)

Purpose: Use the completed symbol tables to verify that all identifiers are properly used, all types are compatible, and all function calls have correct parameters.

Mapping of semantic actions to AST nodes:

AST Node	Method	Effect
`FuncDef`	`visitFuncDef()`	Retrieves function's SymbolTable from funcTableMap. Sets currentScope and currentReturnType. Visits all statements in StatList.
`ProgramBlock`	`visitMainBlock()`	Sets currentScope to main's table. currentReturnType =

		null. Visits non-VarDecl children as statements.
`AssignStat`	`visitStatement()`	Infers LHS and RHS types via inferType(). Reports 10.2 if types differ.
`ReturnStat`	`visitStatement()`	Infers return expression type. Compares against currentReturnType. Reports 10.3 if mismatch.
`IfStat`	`visitStatement()`	Infers condition type. Recursively visits then-block and else-block.
`WhileStat`	`visitStatement()`	Infers condition type. Recursively visits loop body.
`IntNum / FloatNum`	`inferType()`	Returns "int" or "float" directly. Leaf type inference.
`DataMember`	`inferDataMember()`	Looks up variable via lookupVar() scope chain. Reports 11.1 if not found. Checks array dimensions (13.1) and integer indices (13.2). Returns variable type.
`FuncCall`	`inferFuncCall()`	Finds function entry with overload resolution by param count. Reports 11.3/11.4 if not found. Checks param count (12.1) and types (12.2, 13.3). Returns return type.
`Dot`	`inferDot()`	Infers left type. Checks it's a class type (15.1). Resolves right operand in class table. Reports 11.2 (data) or 11.3 (function) if not found.
`AddOp / MultOp`	`inferBinOp()`	Infers both operand types. Reports 10.1 if they differ. Returns the common type for attribute migration.
`RelExpr`	`inferRelExpr()`	Infers operand types (children[0] and children[2], skipping RelOp). Checks type compatibility (10.1).
`Sign / Not`	`inferType()`	Returns child's type unchanged. Unary operators inherit operand type.

2.6. Error Reporting

All semantic errors and warnings are collected from both passes, sorted by source line number, and written to a `.outsemanticerrors` file. Format:

```
[error][8.1] line 7: multiply declared class 'POLYNOMIAL'
[warning][9.3] line 17: Overridden member function 'evaluate'
```

Code	Level	Message
6.1	error	undeclared member function definition
6.2	error	undefined member function declaration
8.1	error	multiply declared class
8.2	error	multiply declared free function
8.3	error	multiply declared data member in class
8.4	error	multiply declared variable in function
8.5	warning	shadowed inherited data member
8.6	warning	local variable in a member function shadows a data member of its class
9.1	warning	Overloaded free function
9.2	warning	Overloaded member function
9.3	warning	Overridden member function
10.1	error	Type error in expression
10.2	error	Type error in assignment statement
10.3	error	Type error in return statement
11.1	error	Undeclared local variable
11.2	error	Undeclared member variable
11.3	error	Undeclared member function
11.4	error	Undeclared/undefined free function
11.5	error	Undeclared class
12.1	error	Function call with wrong number of parameters
12.2	error	Function call with wrong type of parameters

13.1	error	Use of array with wrong number of dimensions
13.2	error	Array index is not an integer
13.3	error	Array parameter using wrong number of dimensions
14.1	error	Circular class dependency
15.1	error	"." operator used on non-class type

3. Use of Tools

There are no tools used for this assignment.